

TradingBot AI - Documentacao Unificada

Resumo oficial baseado no README.md

1. Visao Geral e Arquitetura

- Sistema de trading para memecoins com validacao, analise por regras e execucao automatizada.
- Monorepo com microservicos independentes e frontend Next.js.

Microservicos principais

- API Gateway (porta 4000): autentica, agrega e encaminha requisicoes.
- Validator Service (porta 4001): valida tokens na BSC e avalia riscos.
- Signal Service (porta 4002): gera sinais BUY, HOLD ou SELL com base nas regras.
- Executor Service (porta 4003): executa simulacoes ou opera na testnet.
- Frontend Next.js (porta 3000): interface minimalista para investidores.
- PostgreSQL ou Supabase: banco centralizado para usuarios, tokens e ordens.

2. Setup rapido com Docker

- Requisitos: Docker, Docker Desktop, Docker Compose, arquivo .env na raiz.
- Subir servicos: `docker-compose -f infra/docker-compose.yml up -d`.
- Encerrar: `docker-compose -f infra/docker-compose.yml down`.
- Enderecos: API 4000, Validator 4001, Signal 4002, Executor 4003, Frontend 3000.

3. Setup local sem Docker

- Requisitos: Node.js 20+, PostgreSQL 15+, Git instalado.
- Criar database tradingbot e usuario botuser com permissao completa.
- Aplicar schema: `psql -U botuser -d tradingbot -f infra/postgres/schema.sql`.
- Executar `npm install` em cada servico e no frontend.
- Rodar `npm run dev` em terminais separados ou usar `./start-all.ps1`.

4. Variaveis de ambiente

- Arquivo .env centraliza configuracoes de banco, autenticacao e servicos.
- Principais chaves: `POSTGRES_HOST`, `POSTGRES_PORT`, `DATABASE_URL`, `JWT_SECRET`.
- Configuracoes do bot: `BOT_EXECUTION_MODE`, `BOT_RISK_LEVEL`.
- Frontend utiliza `NEXT_PUBLIC_API_URL` e `CORS_ORIGIN`.
- Suporte Supabase: usar URLs com `sslmode=require` e gerar client Prisma.

5. Endpoints principais

- Autenticacao: `POST /api/auth/register`, `POST /api/auth/login`, `GET /api/auth/me`.
- Validacao: `POST /validate`, `GET /validated-tokens`, `GET /risk/:contractAddress`.
- Tokens: `GET /api/tokens` e `GET /api/tokens/:contractAddress`.
- Sinais: `POST /analyze`, `GET /signals/active`.
- Execucao: `POST /execute/buy`, `POST /execute/sell`, `GET /positions`.

6. Seguranca

- JWT para autenticacao, bcrypt para hash de senhas.
- Criptografia AES-256 para chaves privadas e dados sensiveis.
- Rate limiting, CORS e validacao de input com Zod no API Gateway.

7. Banco de dados

- Tabelas chave: `users`, `user_profiles`, `tokens`, `signals`, `orders`, `ledger_entries`, `audit_logs`.
- Views auxiliares: `open_positions` e `user_performance`.
- Triggers mantem campos `updated_at` sincronizados.

8. Validacao e analise de risco

- Ingestao continua via GeckoTerminal com filtros para memecoins relevantes.
- Consulta a BscScan ou Etherscan para idade do contrato e distribuicao de holders.
- Motor probabilistico gera memecoin_score, risk_score, scam_probability e risk_level.
- Resultados expostos via API Gateway e Validator para o frontend.

9. IA de sinais

- Regras ponderam volume 24h, liquidez, numero de holders, idade e score de seguranca.
- Sinais BUY, HOLD ou SELL incluem confianca percentual e multiplicador estimado.
- Evita duplicidade atualizando o ultimo sinal quando variacoes sao pequenas.

10. Execucao de trades

- Modos simulation e live (BSC Testnet) com selecao por variavel de ambiente.
- Perfis de risco: conservador 1%, moderado 3%, agressivo 6% por operacao.
- Integracao com PancakeSwap Router para ordens e registro em ledger.

11. Diagnostico e troubleshooting

- Erro 500 ao registrar usuario: checar banco ativo, variaveis .env e logs inicial.
- Reaplicar schema com psql se necessario.
- Testar conexao com script Node que executa SELECT NOW().
- Conflito na porta 5432: projeto padrao 5433, ajustar .env se mudar porta.

12. Tecnologias principais

- Backend: Node.js, Express, TypeScript.
- Blockchain: ethers.js e web3.js.
- Database: PostgreSQL, Supabase e Prisma para schema.
- Frontend: Next.js 14, React, Tailwind CSS.
- Infra: Docker, Docker Compose e scripts PowerShell.

13. Plano de implementacao MVP

- Estrutura de pastas com services, web/frontend, infra e shared/types.
- Fases: ingestao + validacao, IA de sinais, executor, API Gateway, frontend.
- To-dos referencia: monorepo, schema, docker compose, autenticacao, testnet, UI.

14. Licenca e contribuicao

- Licenca MIT e contribuicoes abertas via pull request.

15. Contato e suporte

- Consultar secao de diagnostico ou abrir issue com contexto, logs e passos.

Anexos e scripts utilitarios

- start-all.ps1 inicia todos os servicos carregando o .env global.
- Infra/kubernetes contem manifests para deploy em cloud.
- secret-example.yaml e ingress.yaml servem de base para configuracoes finais.

Observacoes finais

- Documento reflete o estado do README em 13/11/2025.
- Pode ser usado em apresentacoes para clientes, parceiros ou equipe interna.